

コンピュータアーキテクチャI

担当：佐々木 敬泰

第11回

乗算アルゴリズムと乗算器

今日の内容

- 乗算アルゴリズム
- 乗算器の構成

コンピュータアーキテクチャI

3

筆算

- 1234 x 5678 = ?

7006652

				1	2	3	4
x				5	6	7	8
				9	8	7	2
		8		6	3	8	
.	.	.	略	.	.	.	
7	0	0	6	6	5	2	

二進数の乗算も同じ

乗算器のアルゴリズム(1/2)

- 部分積を累算する

				1	1	0	1
x				0	1	0	1
				1	1	0	1
			0	0	0	0	
		1	1	0	1		
	0	0	0	0			
0	1	0	0	0	0	0	1

- PROD = 1101
- PROD = PROD + 00000
- PROD = PROD + 110100
- PROD = PROD + 0000000

コンピュータアーキテクチャI

7

シラバス

1. イントロダクション
2. コンピュータの構成
3. 数の表現
4. 様々なプロセッサと命令セット
5. 基本命令
6. メモリアクセス命令
7. 分岐命令
8. 高級言語からアセンブラへの変換
9. プログラムの翻訳と起動
10. 加減算
11. 乗算アルゴリズムと乗算器
12. 除算アルゴリズムと除算器
13. 浮動小数点
14. 浮動小数点の乗算と誤差
15. 定期試験

コンピュータアーキテクチャI

2

乗算アルゴリズム

コンピュータアーキテクチャI

4

乗算

- $13_{10} * 5_{10} = 65_{10}$

- $1101_2 * 0101_2 = 01000001_2$

				1	1	0	1
x				0	1	0	1
				1	1	0	1
			0	0	0	0	
		1	1	0	1		
	0	0	0	0			
0	1	0	0	0	0	0	1

乗算器のアルゴリズム(2/2)

- 被乗数 × 乗数 = 積
- まず、積 = 0 とする
- 乗数の桁数回、以下を繰り返す
 - 乗数の下位1桁が1なら積 = 積 + 被乗数
 - 乗数を1桁右にずらす (右シフト)
 - 被乗数を1桁左にずらす (左シフト)

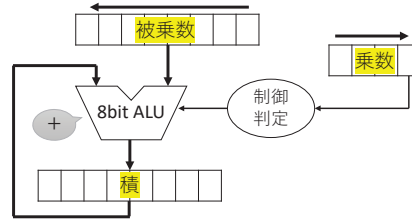
コンピュータアーキテクチャI

8

乗算器の構成

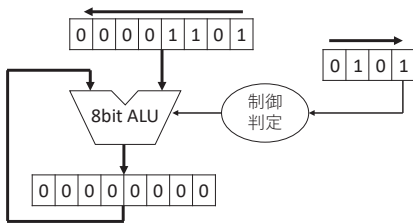
4bit × 4bitの8bit乗算器

- $13_{10} * 5_{10} = 65_{10}$
- $1101_2 * 0101_2 = 01000001_2$



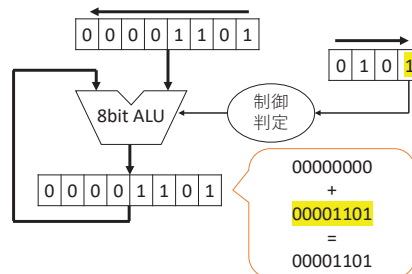
乗算器(0/4)

- $13_{10} * 5_{10} = 65_{10}$
- $1101_2 * 0101_2 = 01000001_2$



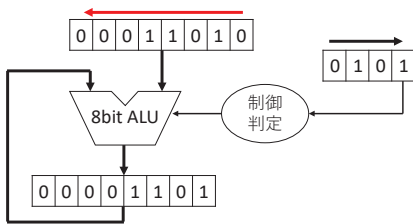
乗算器(1A/4)

- $13_{10} * 5_{10} = 65_{10}$
- $1101_2 * 0101_2 = 01000001_2$



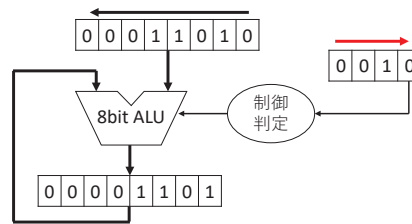
乗算器(1B/4)

- $13_{10} * 5_{10} = 65_{10}$
- $1101_2 * 0101_2 = 01000001_2$



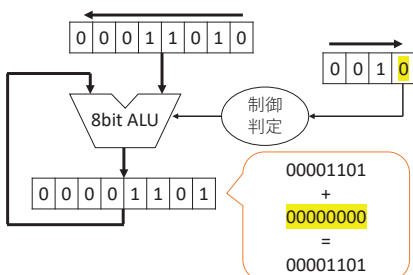
乗算器(1C/4)

- $13_{10} * 5_{10} = 65_{10}$
- $1101_2 * 0101_2 = 01000001_2$



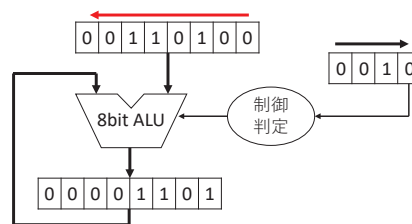
乗算器(2A/4)

- $13_{10} * 5_{10} = 65_{10}$
- $1101_2 * 0101_2 = 01000001_2$



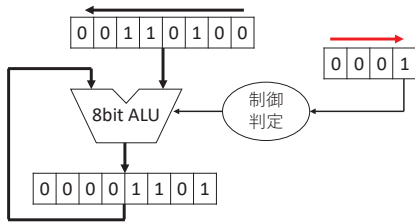
乗算器(2B/4)

- $13_{10} * 5_{10} = 65_{10}$
- $1101_2 * 0101_2 = 01000001_2$



乗算器(2C/4)

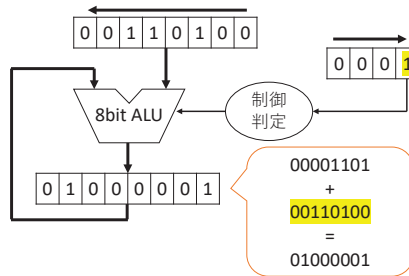
- $13_{10} * 5_{10} = 65_{10}$
- $1101_2 * 0101_2 = 01000001_2$



17

乗算器(3A/4)

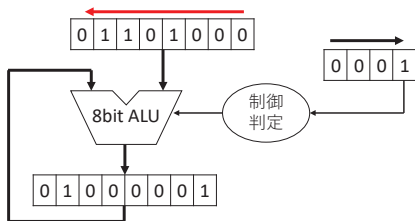
- $13_{10} * 5_{10} = 65_{10}$
- $1101_2 * 0101_2 = 01000001_2$



18

乗算器(3B/4)

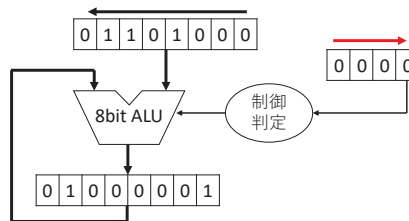
- $13_{10} * 5_{10} = 65_{10}$
- $1101_2 * 0101_2 = 01000001_2$



19

乗算器(3C/4)

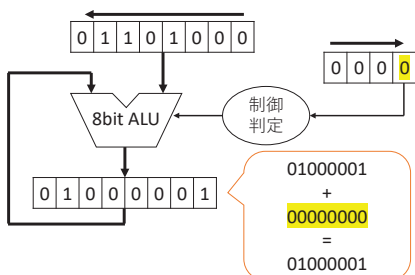
- $13_{10} * 5_{10} = 65_{10}$
- $1101_2 * 0101_2 = 01000001_2$



20

乗算器(4A/4)

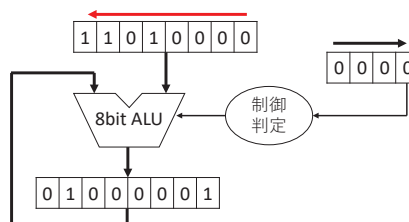
- $13_{10} * 5_{10} = 65_{10}$
- $1101_2 * 0101_2 = 01000001_2$



21

乗算器(4B/4)

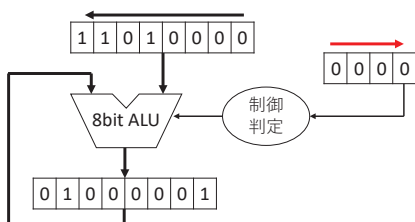
- $13_{10} * 5_{10} = 65_{10}$
- $1101_2 * 0101_2 = 01000001_2$



22

乗算器(4C/4)

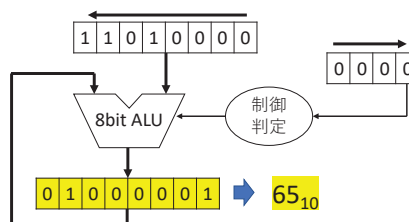
- $13_{10} * 5_{10} = 65_{10}$
- $1101_2 * 0101_2 = 01000001_2$



23

乗算器(4C/4)

- $13_{10} * 5_{10} = 65_{10}$
- $1101_2 * 0101_2 = 01000001_2$



24

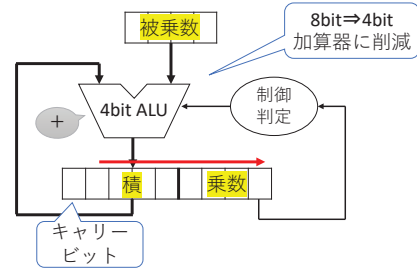
考察

- $4\text{bit} \times 4\text{bit}$ の乗算に、
 - 4bit レジスタ $\times 1$ 本
 - 8bit レジスタ $\times 2$ 本
 - 8bit 演算器
 が必要。しかし、使われていない部分が多い。そこで、乗算器の改良を行う。
- アイディア
 - 乗数は下位 1bit のみ必要で、ステップが進むにつれて下位ビットは不要になる。
 - 積の上位ビットはステップの最初は 0 になる。
 - 積と乗数でレジスタを共有する。
 - 加算器も 4bit で済む。

25

乗算器の改良

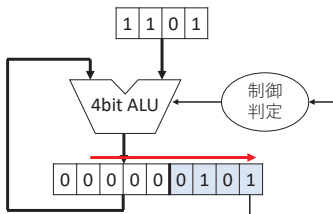
- $13_{10} * 5_{10} = 65_{10}$
- $1101_2 * 0101_2 = 01000001_2$



26

乗算器の改良(0/4)

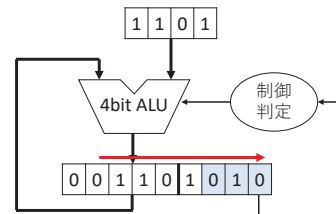
- $13_{10} * 5_{10} = 65_{10}$
- $1101_2 * 0101_2 = 01000001_2$



27

乗算器の改良(1/4)

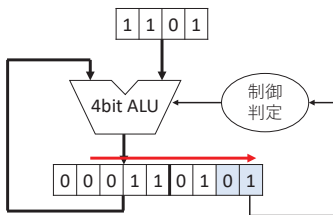
- $13_{10} * 5_{10} = 65_{10}$
- $1101_2 * 0101_2 = 01000001_2$



28

乗算器の改良(2/4)

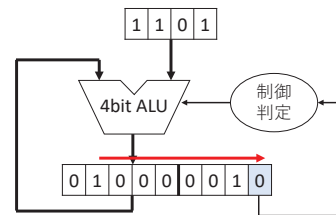
- $13_{10} * 5_{10} = 65_{10}$
- $1101_2 * 0101_2 = 01000001_2$



29

乗算器の改良(3/4)

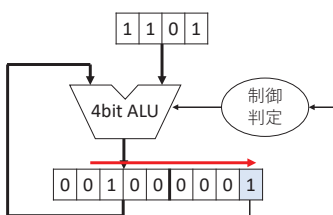
- $13_{10} * 5_{10} = 65_{10}$
- $1101_2 * 0101_2 = 01000001_2$



30

乗算器の改良(4/4)

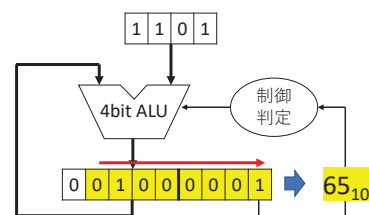
- $13_{10} * 5_{10} = 65_{10}$
- $1101_2 * 0101_2 = 01000001_2$



31

乗算器の改良(4/4)

- $13_{10} * 5_{10} = 65_{10}$
- $1101_2 * 0101_2 = 01000001_2$



32

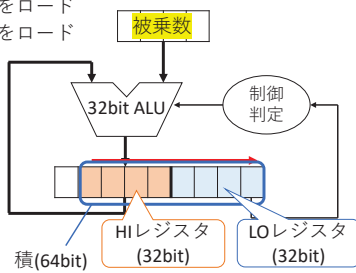
MIPSプロセッサと乗算器

• MIPS命令

mult \$2, \$3 # 2×3

mfhi \$9 # 上位32bitをロード

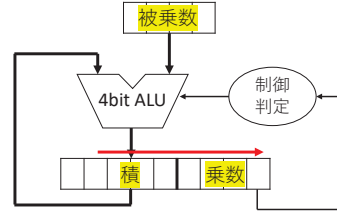
mflo \$8 # 下位32bitをロード



33

練習

• 8bit (4bit × 4bit) の乗算器で $6_{10} \times 7_{10}$ を計算



$$6_{10} \times 7_{10} = 42_{10}$$

$$0110_2 \times 0111_2 = 00101010_2$$

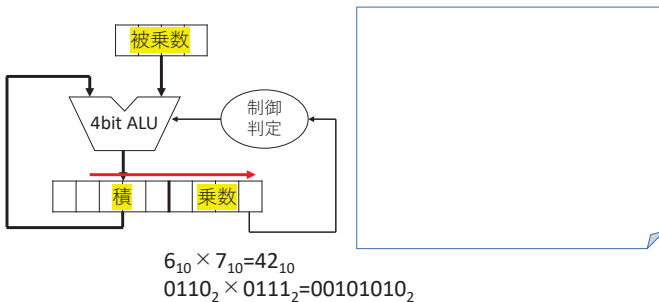
Step	被乗数	積
0	0110	
1	0110	
2	0110	
3	0110	
4	0110	

コンピュータアーキテクチャI

34

練習の答え

• 8bit (4bit × 4bit) の乗算器で $6_{10} \times 7_{10}$ を計算



$$6_{10} \times 7_{10} = 42_{10}$$

$$0110_2 \times 0111_2 = 00101010_2$$

コンピュータアーキテクチャI

35

符号付き整数の乗算

• $-3 \times 3 = -9$

• $1101_2 \times 0011_2 = 11110111_2$

• 1101を符号拡張する

				1	1	0	1
x				0	0	1	1
	1	1	1	1	1	0	1
	1	1	1	1	0	1	
	0	0	0	0	0		
	0	0	0	0			
	1	1	1	1	0	1	1

コンピュータアーキテクチャI

36

符号付き整数の乗算

• $3 \times -3 = -9$

• $0011_2 \times 1101_2 = 11110111_2$

• 乗数が負の時は???

				0	0	1	1
x				1	1	0	1
				0	0	1	1
			0	0	0	0	
		0	0	1	1		
	0	0	1	1			
	0	1	0	0	1	1	1

コンピュータアーキテクチャI

37

符号付き整数の乗算

• 解決方法

1. 演算前に正数にして、演算後に符号を調整
2. 直接乗算法
3. Booth法を用いる

コンピュータアーキテクチャI

38

直接乗算法で負数を考慮する

				-a3	+a2	+a1	+a0
			*)	-b3	+b2	+b1	+b0
+))				-p30	+p20	+p10	+p00
+))				-p31	+p21	+p11	+p01
+))				-p31	+p22	+p12	+p02
+))				+p33	-p23	-p13	-p03
				整理すると↓			
+))				1	~p30	p20	p10
+))				~p31	p21	p11	p01
+))				~p31	p22	p12	p02
+))	1	p33	~p23	~p13	~p03		

$$p_{ij} = a_i \times b_j$$

詳細はGoogle[二の補数表現 乗算器][検索]

コンピュータアーキテクチャI

39

Booth法 (ブースの乗算アルゴリズム)

• 基本的なアイディア

$$123 \times 9999 = 123 \times (10000 - 1) = 123 \times 10000 - 123$$

• 利点

- 部分積の加算数が減る (場合がある)
- 2の補数表現の乗算に対応している

コンピュータアーキテクチャI

40

ブース法（4bitの場合）

- $4 \times 6 = 24$ を考える
- $0100_2 \times 0110_2 = 00011000_2$
- Pを4bit×2+1ビットのレジスタとする
 - 下位に1bitおまけが付く
- 手順
 1. P（9bitのレジスタ）に乗数(0110₂)を入れる
 2. Pを左に1bitシフト
 3. Pの下位2bit（P₁P₀）の値により以下の処理をする
 - 00：何もしない
 - 01：Pに（被乗数×32）を加算
 - 10：Pに（被乗数×32）を減算
 - 11：何もしない
 4. Pを1bit算術右シフト
 5. 手順3に戻る（4回繰り返す）

ブース法（4bitの場合）

- $4 \times 6 = 24$ を考える
- $0100_2 \times 0110_2 = 00011000_2$

手順1, 2実行直後

	8	7	6	5	4	3	2	1	0	
0	0	0	0	0	0	1	1	0	0	右シフト
1	0	0	0	0	0	0	1	1	0	減算→右シフト
-)	0	1	0	0						
	1	1	0	0	0	0	1	1		
3	1	1	1	0	0	0	0	1	1	右シフト
	1	1	1	1	0	0	0	0	1	加算→右シフト
+	0	1	0	0						
	0	0	1	1	0	0	0	0		
4	0	0	0	1	1	0	0	0	0	

ブース法（4bitの場合）

- $4 \times -6 = -24$ を考える
- $0100_2 \times 1010_2 = 11101000_2$

	8	7	6	5	4	3	2	1	0	
0	0	0	0	0	1	0	1	0	0	右シフト
1	0	0	0	0	0	1	0	1	0	減算→右シフト
-)	0	1	0	0						
	1	1	0	0	0	1	0	1		
3	1	1	1	0	0	0	1	0	1	加算→右シフト
+	0	1	0	0						
	0	0	1	0	0	0	1	0		
4	0	0	0	1	0	0	0	1	0	減算→右シフト

ブース法（4bitの場合）【続き】

- $4 \times -6 = -24$ を考える
- $0100_2 \times 1010_2 = 11101000_2$

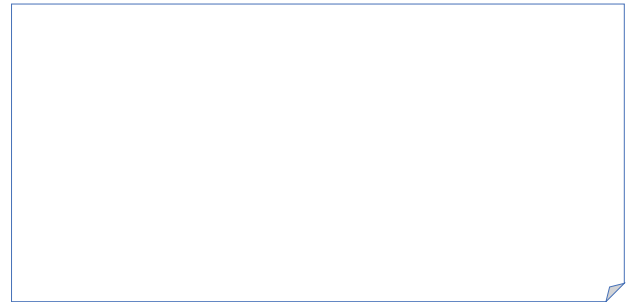
	8	7	6	5	4	3	2	1	0	
4	0	0	0	1	0	0	0	1	0	減算→右シフト
-)	0	1	0	0						
	1	1	0	1	0	0	0	1		
	1	1	1	0	1	0	0	0	1	

ブース法の練習

- $3 \times -4 = -12$
- $0011_2 \times 1100_2 = 11110100_2$
- 手順

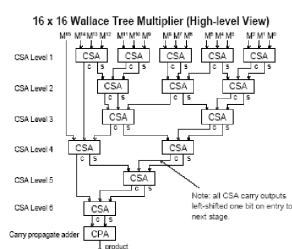
ブース法の練習の答え

- $3 \times -4 = -12$
- $0011_2 \times 1100_2 = 11110100_2$
- 手順



Wallace Tree

- 桁上げ保存加算器(CSA : Carry Save Adder)をツリー上に配置した乗算器
 - ワレスツリー
 - フラスツリー
 - ウォレスツリー
- 2の補数表現にも対応
- 実際の商用CPUでも利用



https://www.researchgate.net/figure/Typical-Wallace-tree_fig8_259590209